PRAKTIKUM TEKNOLOGI IOT II

LAPORAN PRAKTIK

TEKNOLOGI IOT II

Disusun dalam rangka memenuhi salah satu syarat kelengkapan nilai Praktikum Mata Kuliah Teknologi IOT II

DISUSUN OLEH:

NAMA : Tengku Geisya Gustiana Syah

NPM : 25782028

KELAS :3A

PRODI : TEKNOLOGI REKAYASA INTERNET

Dosen :

Dr. Ir. Septafiansyah Dwi Putra, S. T., M.T.

Agus Ambarwari, S.Pd.,M.kom,

TEKNOLOGI REKAYASA INTERNET

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI LAMPUNG

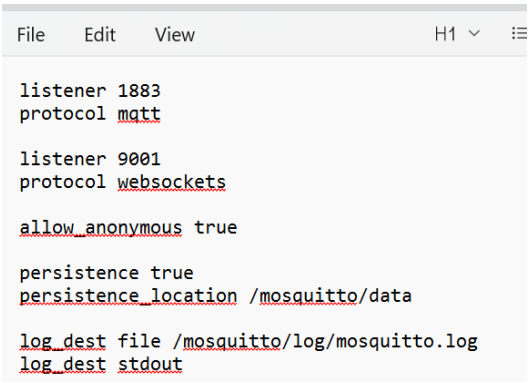
2026

7.5 Praktikum

7.5.1 Praktikum 1: Pembangunan Ekosistem IoT Server dengan Docker Compose

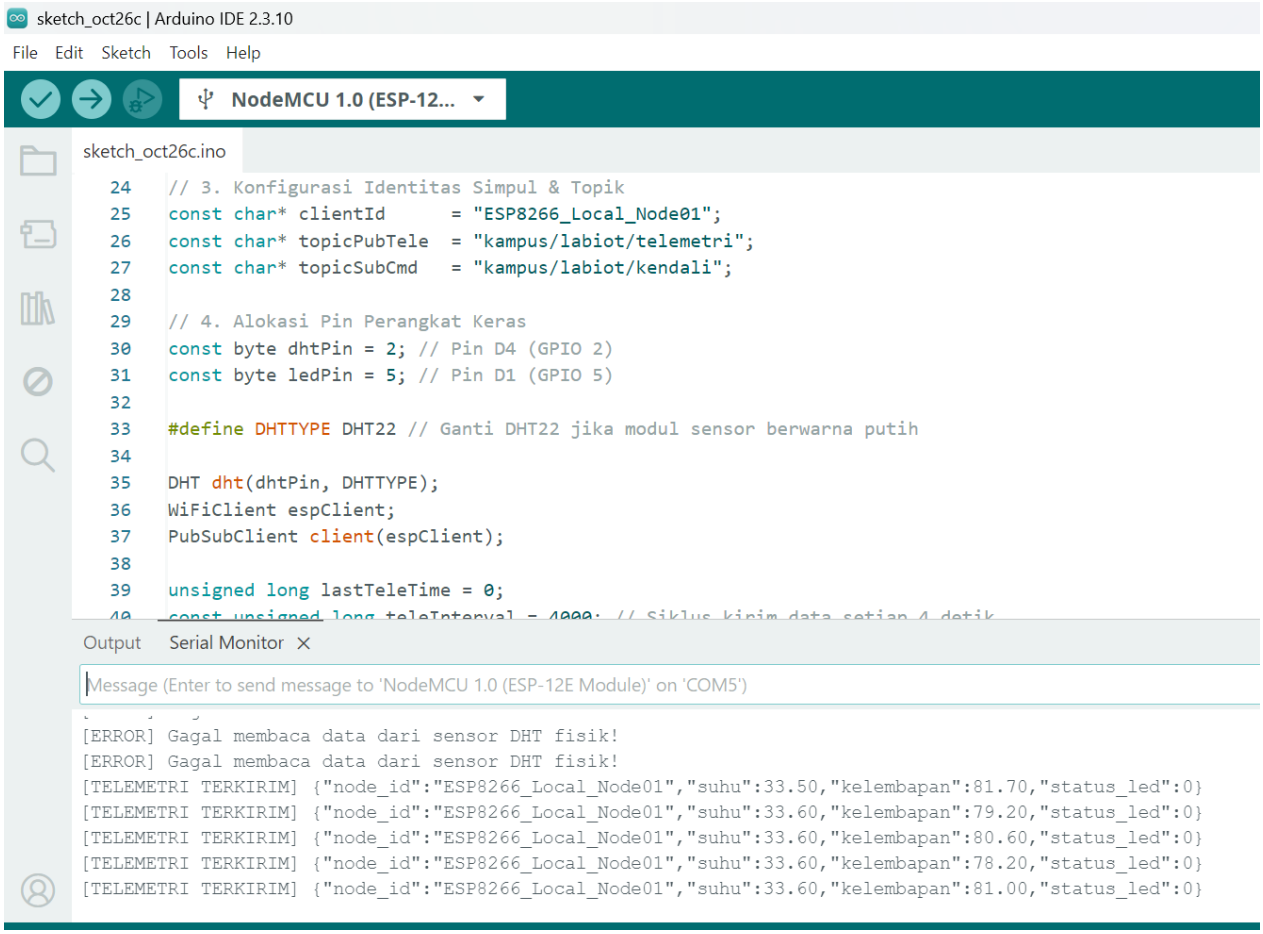


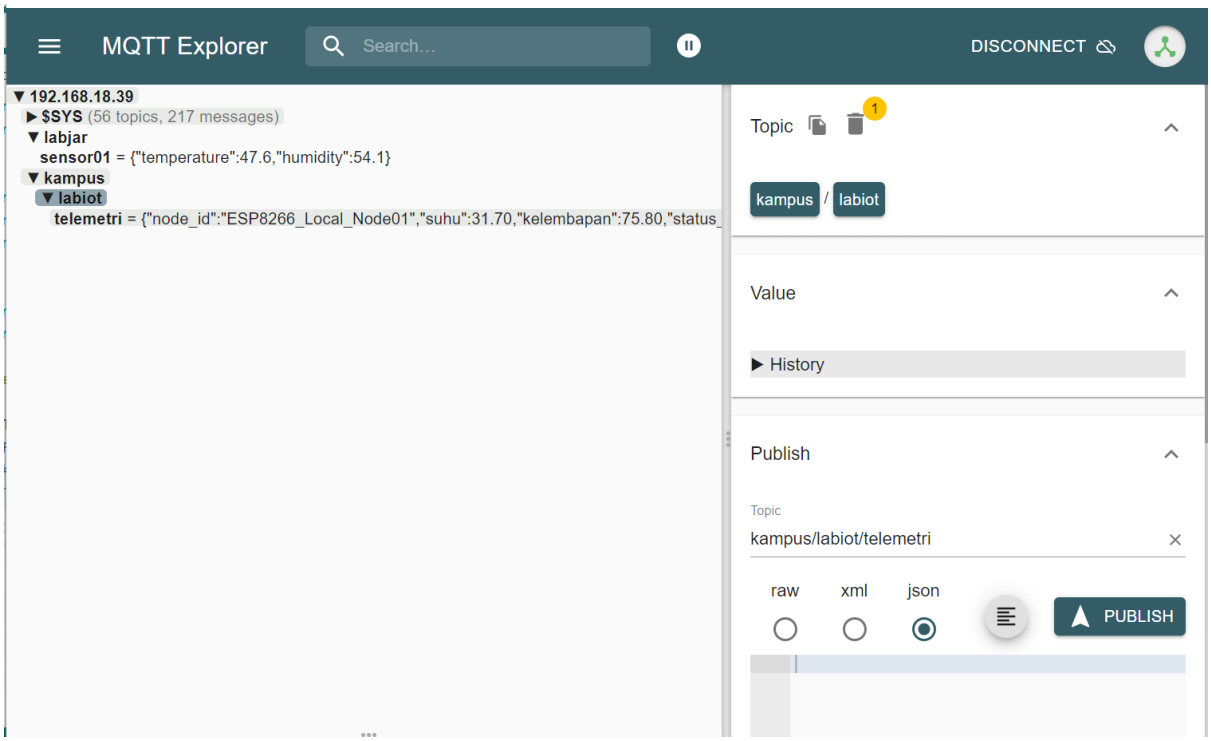
Pada praktikum ini dilakukan pembangunan server IoT lokal menggunakan Docker Compose. Tahap pertama dimulai dengan membuat direktori kerja yang digunakan untuk menyimpan file konfigurasi Mosquitto dan Node-RED. Selanjutnya, dibuat file mosquitto.conf untuk mengatur komunikasi MQTT melalui port 1883 dan WebSockets melalui port 9001.

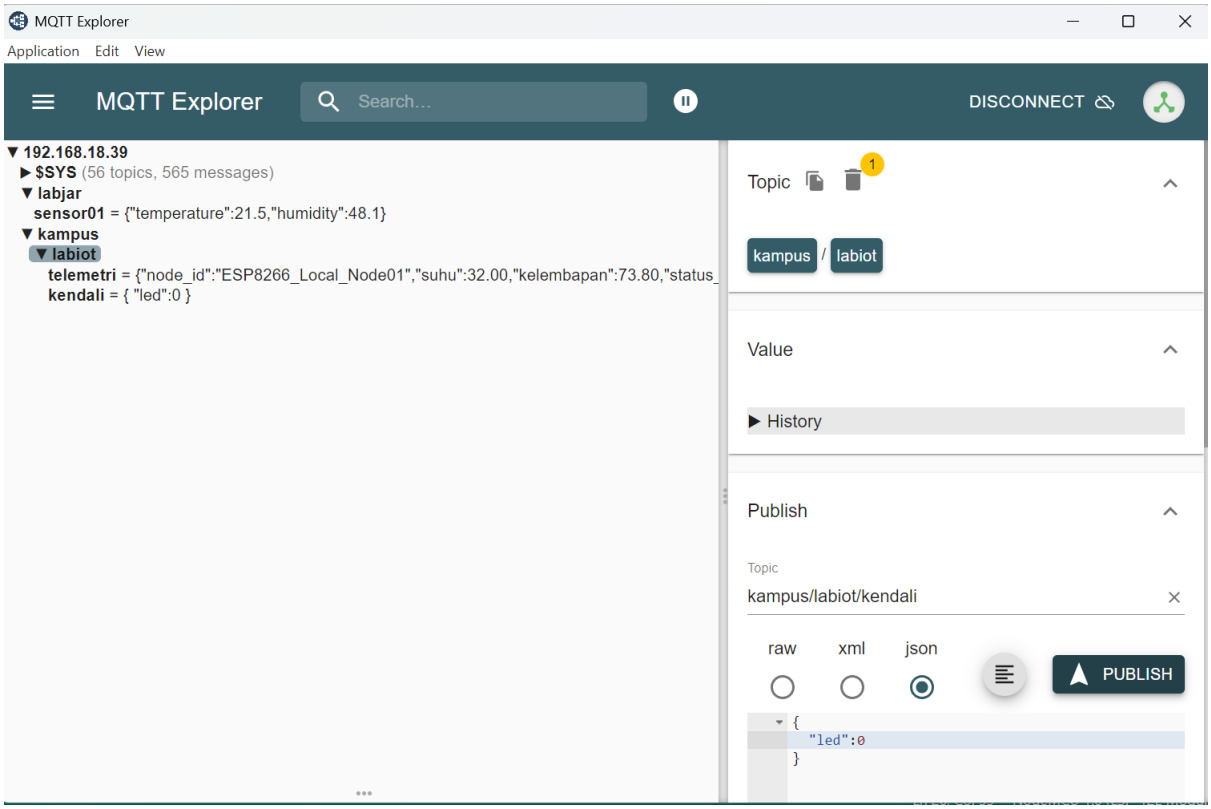


Setelah itu, dibuat file docker-compose.yml yang berisi konfigurasi dua layanan, yaitu Eclipse Mosquitto sebagai MQTT Broker dan Node-RED sebagai platform pengolahan data. Kedua layanan dijalankan menggunakan perintah docker compose up -d. Keberhasilan praktikum diperiksa melalui perintah docker ps untuk memastikan kedua container berjalan dengan baik. Kemudian, antarmuka Node-RED diakses melalui browser menggunakan alamat http://localhost:1880.

7.5.2 Praktikum 2: Migrasi Simpul NodeMCU ESP8266 ke Broker MQTT Lokal








```
----------------------------------------
[INSTRUKSI MASUK] Topik: kampus/labiot/kendali
[PAYLOAD]         : {
  "led":0
}
--> [AKTUATOR] Status LED diubah: PADAM (OFF)
----------------------------------------
```

Pada praktikum ini dilakukan proses migrasi koneksi NodeMCU ESP8266 dari broker MQTT publik menuju broker MQTT lokal yang telah dibangun menggunakan Eclipse Mosquitto pada Docker. Migrasi ini bertujuan agar komunikasi data antara perangkat mikrokontroler dan server dapat dilakukan melalui jaringan lokal tanpa harus bergantung pada broker publik.

Rangkaian perangkat keras yang digunakan terdiri dari NodeMCU ESP8266, sensor DHT11/DHT22, dan LED sebagai indikator aktuator. Sensor DHT digunakan untuk membaca kondisi lingkungan berupa suhu dan kelembapan. Sensor tersebut dihubungkan ke pin D4 (GPIO 2), sedangkan LED dihubungkan ke pin D1 (GPIO 5) dengan tambahan resistor 220 Ω sebagai pembatas arus. Pin VCC sensor dihubungkan ke 3V3 dan pin GND dihubungkan ke GND NodeMCU.

Tahap pertama adalah mengetahui alamat IPv4 laptop yang digunakan sebagai server MQTT lokal. Pada sistem operasi Windows, alamat tersebut diperiksa melalui Command Prompt atau PowerShell menggunakan perintah ipconfig. Alamat IPv4 pada bagian Wireless LAN adapter Wi-Fi kemudian dicatat karena akan digunakan sebagai alamat broker MQTT pada program NodeMCU.

Selanjutnya dilakukan pemeriksaan konfigurasi Windows Defender Firewall. Firewall perlu mengizinkan koneksi masuk melalui port TCP 1883 agar NodeMCU dapat berkomunikasi dengan broker Mosquitto yang berjalan pada laptop. Jika koneksi melalui port tersebut diblokir, perangkat tidak dapat terhubung meskipun sudah menggunakan alamat IP broker yang benar.

Tahap berikutnya adalah melakukan pemrograman pada Arduino IDE dengan menggunakan beberapa library, yaitu ESP8266WiFi, PubSubClient, ArduinoJson, dan DHT. Library tersebut memiliki fungsi masing-masing, yaitu menghubungkan NodeMCU ke jaringan Wi-Fi, mengatur komunikasi MQTT, membentuk data dalam format JSON, serta membaca data dari sensor DHT.

Pada program, terlebih dahulu dilakukan pengaturan nama dan kata sandi Wi-Fi sesuai jaringan yang digunakan. Setelah itu, alamat broker MQTT diubah menggunakan alamat IPv4 laptop, dengan port komunikasi 1883. Pengaturan ini bertujuan agar NodeMCU tidak lagi mengirimkan data ke broker publik, melainkan langsung menuju server Mosquitto lokal.

Program juga mengatur identitas perangkat menggunakan clientId serta menentukan topik komunikasi MQTT. Topik kampus/labiot/telemetri digunakan untuk mengirimkan data sensor dari NodeMCU menuju broker, sedangkan topik kampus/labiot/kendali digunakan untuk menerima pesan kendali dari server. Dengan pembagian topik tersebut, komunikasi antara perangkat dan server dapat berlangsung secara dua arah.

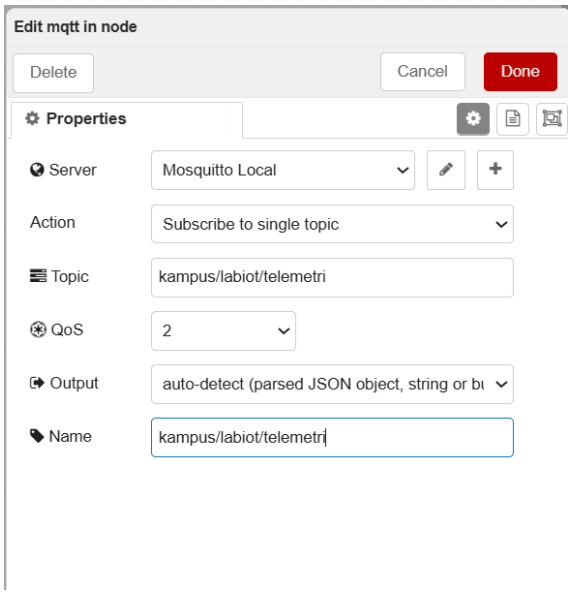
Setelah seluruh konfigurasi selesai, program diunggah ke board NodeMCU ESP8266. Kemudian Serial Monitor dibuka menggunakan kecepatan komunikasi 115200 baud untuk memantau proses koneksi perangkat. Jika konfigurasi berhasil, NodeMCU akan memperoleh alamat IP dari jaringan Wi-Fi dan menampilkan pesan [MQTT] Menghubungkan ke Broker Lokal... Terhubung!.

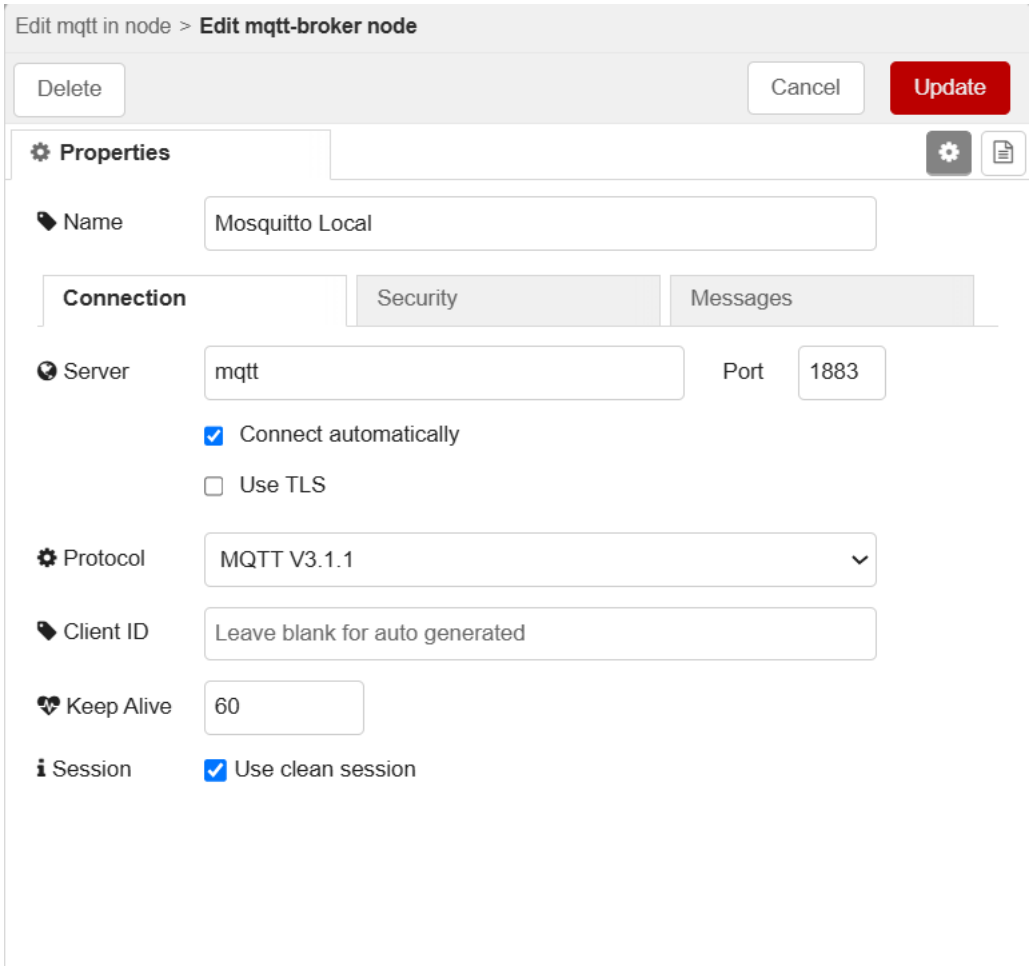
Setelah berhasil terhubung, NodeMCU mulai membaca nilai suhu dan kelembapan dari sensor DHT. Data hasil pembacaan kemudian disusun dalam format JSON dan dikirimkan secara berkala ke broker Mosquitto melalui topik telemetri. Selain mengirimkan data, NodeMCU juga berlangganan pada topik kendali agar dapat menerima instruksi dari server untuk mengatur kondisi LED.

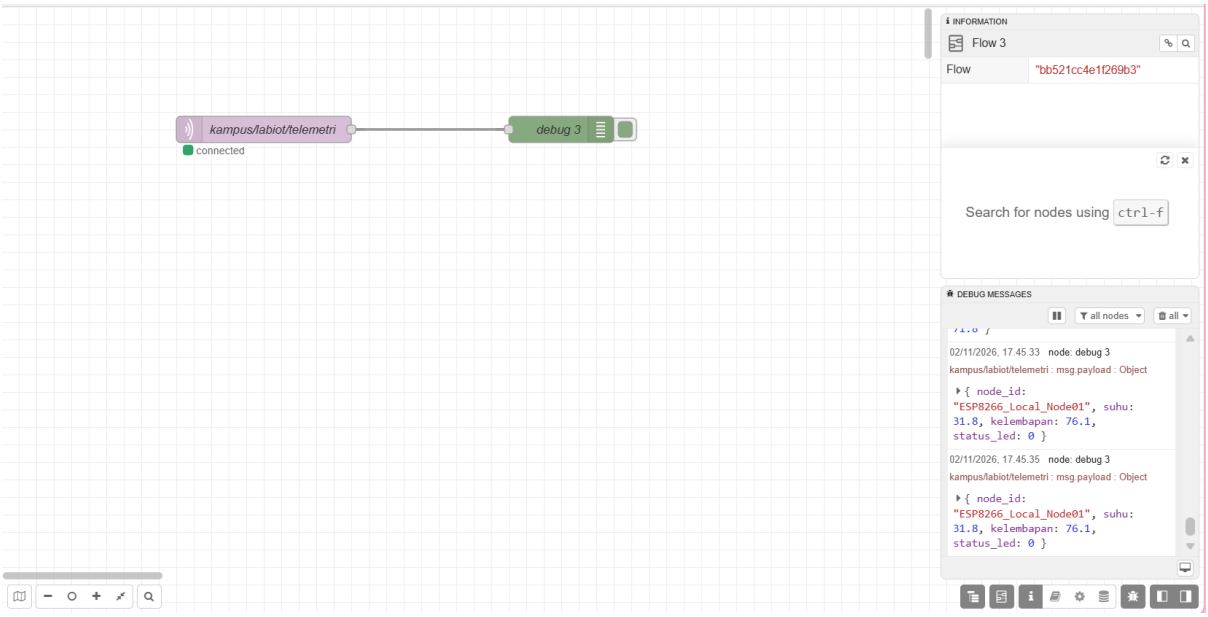
Dengan demikian, komunikasi yang terbentuk tidak hanya berupa pengiriman data sensor, tetapi juga penerimaan perintah dari server menuju perangkat. Hasil praktikum ini menjadi dasar untuk mengintegrasikan data telemetri dan sistem kendali otomatis menggunakan Node-RED pada praktikum berikutnya.

Hasil yang diharapkan: NodeMCU ESP8266 berhasil terhubung ke jaringan Wi-Fi dan broker Mosquitto lokal, dapat mengirimkan data suhu serta kelembapan dalam format JSON, dan dapat menerima pesan kendali untuk mengatur LED melalui komunikasi MQTT dua arah

7.5.3 Praktikum 3: Integrasi Alur Data Telemetri Dua Arah pada Node-RED







Pada praktikum ini dilakukan integrasi antara broker MQTT lokal Mosquitto dengan aplikasi Node-RED untuk memantau dan mengelola aliran data telemetri dari NodeMCU ESP8266. Node-RED digunakan sebagai platform pemrograman visual yang memungkinkan proses penerimaan, pengolahan, dan pengiriman data dilakukan melalui susunan node yang saling terhubung.

Tahap pertama dilakukan dengan membuka antarmuka Node-RED melalui browser menggunakan alamat http://localhost:1880. Setelah halaman kanvas Node-RED terbuka, dilakukan penambahan node mqtt in dari kategori network. Node ini berfungsi menerima pesan yang dikirimkan oleh NodeMCU melalui broker MQTT.

Selanjutnya dilakukan konfigurasi server MQTT dengan membuat koneksi baru bernama Mosquitto Local. Pada bagian server diisi dengan nama layanan mqtt dan port 1883. Penggunaan nama mqtt bertujuan agar kontainer Node-RED dapat menemukan kontainer broker Mosquitto melalui jaringan internal Docker. Alamat localhost tidak digunakan karena akan mengarah ke kontainer Node-RED itu sendiri, bukan ke kontainer broker.

Pada bagian konfigurasi topik, diisi kampus/labiot/telemetri sebagai topik untuk menerima data sensor. Format keluaran diatur menjadi a parsed JSON object agar pesan JSON yang diterima dapat langsung diubah menjadi objek JavaScript. Dengan begitu, nilai suhu dan kelembapan dapat dibaca dan diolah oleh node berikutnya tanpa perlu melakukan proses penguraian data secara manual.

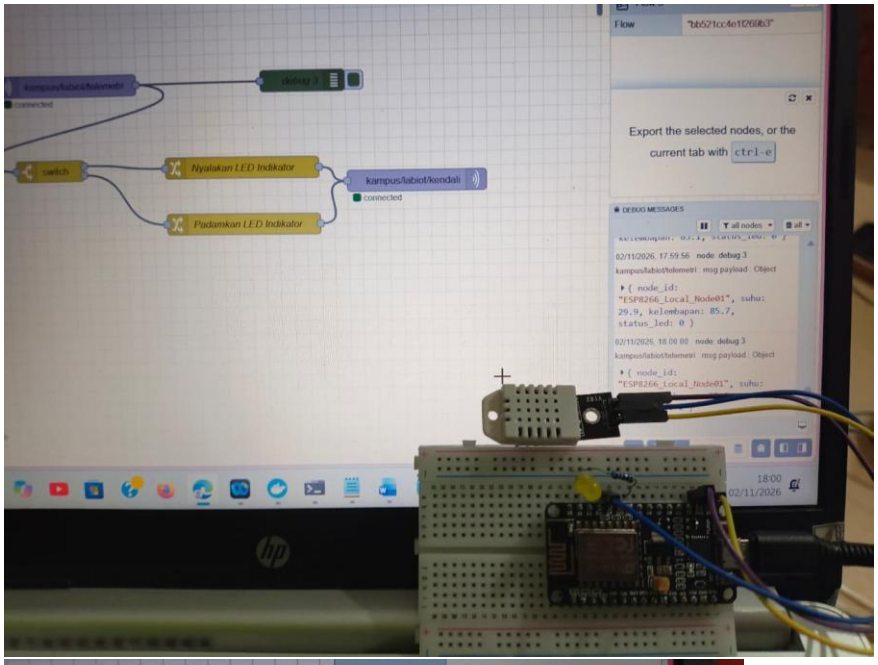
Setelah konfigurasi mqtt in selesai, ditambahkan node debug dari kategori common ke dalam kanvas. Node debug dihubungkan dengan node mqtt in menggunakan garis penghubung (wire). Node ini berfungsi menampilkan pesan yang diterima sehingga isi data telemetri dapat diamati melalui panel Debug Messages pada Node-RED.

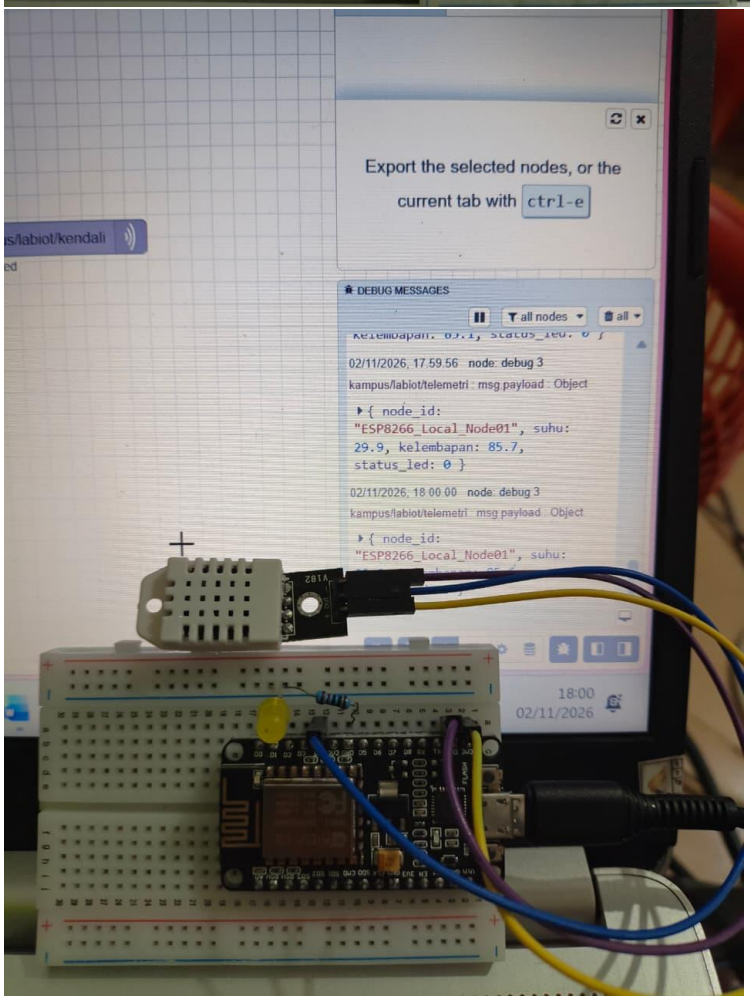
Setelah seluruh node terhubung, dilakukan penerapan alur kerja dengan menekan tombol Deploy pada bagian kanan atas antarmuka. Apabila konfigurasi berhasil, node mqtt in akan menampilkan indikator berwarna hijau dengan status connected. Status tersebut menunjukkan bahwa Node-RED telah berhasil terhubung dengan broker Mosquitto lokal dan siap menerima pesan MQTT.

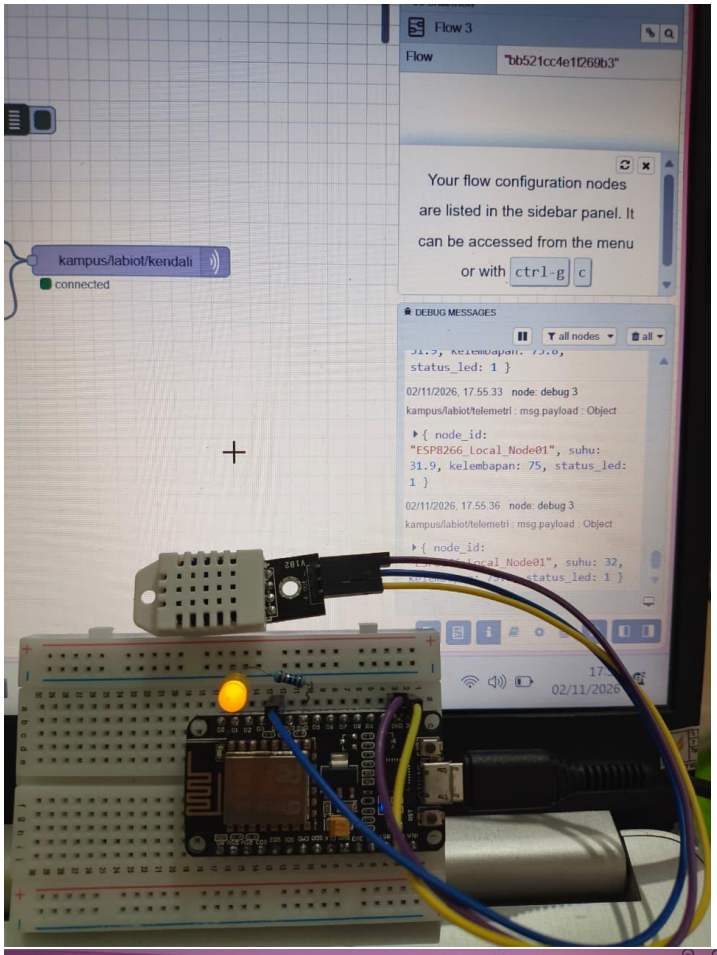
Tahap selanjutnya adalah melakukan pengamatan terhadap data yang masuk melalui panel Debug Messages. Panel tersebut dapat dibuka dengan memilih ikon Debug pada bagian sidebar sebelah kanan. Data telemetri dari NodeMCU akan ditampilkan secara berkala, yaitu setiap 4 detik, dalam bentuk objek JSON yang berisi informasi seperti node_id, suhu, kelembapan, dan status_led.

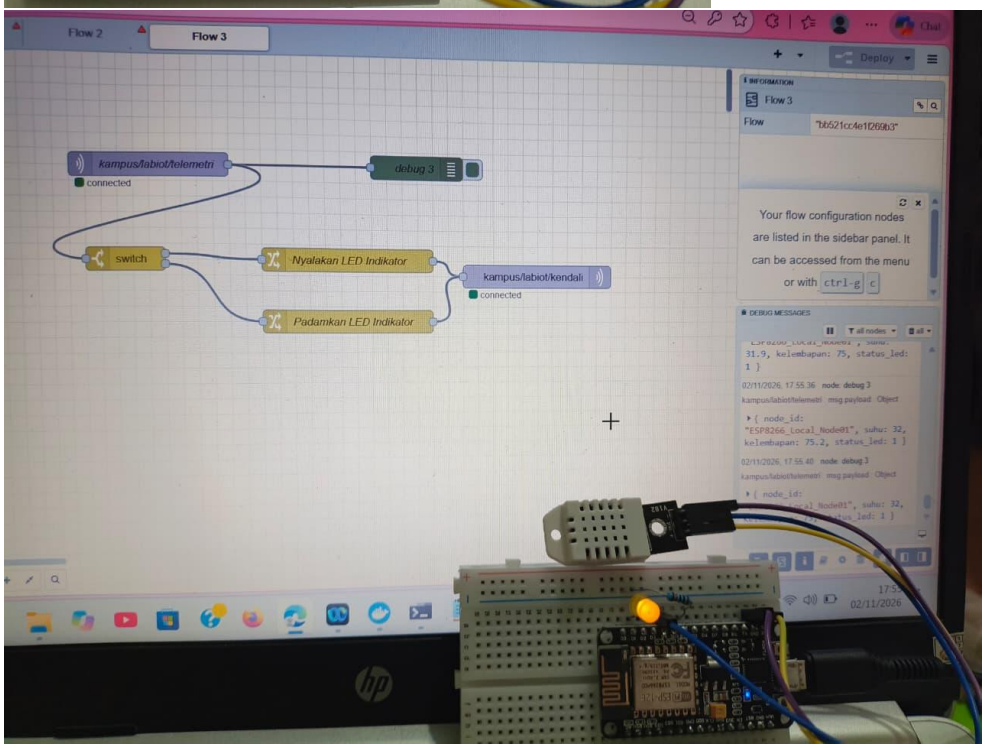
Melalui tampilan tersebut, setiap nilai pada objek JSON dapat diperiksa dengan menekan tanda panah di samping bagian Object. Dengan demikian, mahasiswa dapat mengetahui apakah data sensor berhasil dikirimkan oleh NodeMCU dan diterima oleh Node-RED melalui broker MQTT lokal.

7.5.4 Praktikum 4: Implementasi Rule Engine Otomasi Aktuator Loop Tertutup









Pada praktikum ini dilakukan penerapan sistem otomasi menggunakan Rule Engine pada Node-RED. Sistem dirancang agar LED dapat menyala atau mati secara otomatis berdasarkan nilai suhu yang dibaca oleh sensor DHT.

Proses dimulai dengan menambahkan node switch untuk memeriksa nilai suhu menggunakan dua kondisi, yaitu suhu lebih dari 31°C dan suhu kurang dari atau sama dengan 31°C. Selanjutnya, digunakan dua node change untuk mengatur pesan kendali dalam format JSON. Jika suhu melebihi 31°C, sistem akan mengirimkan perintah {"led": 1} sehingga LED menyala. Sebaliknya, jika suhu berada pada atau di bawah 31°C, sistem mengirimkan perintah {"led": 0} sehingga LED padam.

Kedua node change kemudian dihubungkan ke node mqtt out dengan topik kampus/labiot/kendali. Setelah dilakukan Deploy, sistem diuji dengan mengamati perubahan suhu pada sensor. Hasilnya, LED dapat dikendalikan secara otomatis sesuai dengan kondisi suhu yang terbaca.

7.6 Latihan

Kerjakan latihan berikut secara mandiri berbasis analisis teori dan eksperimen teknis untuk melengkapi laporan praktikum Anda:

- **Latihan 1 (Analisis Jaringan Internal Docker & Service Discovery):**
  Pada Praktikum 3, di dalam konfigurasi node mqtt in pada Node-RED, kita mengisi alamat server broker dengan nama teks **mqtt**, bukan alamat IP lokal laptop (192.168.1.15) maupun localhost.

  i. Mengapa penulisan localhost pada Node-RED di dalam kontainer Docker akan menyebabkan kegagalan koneksi (*Connection Refused*)?

  Jawaban: Penggunaan localhost pada Node-RED menyebabkan kegagalan koneksi karena alamat tersebut mengarah ke kontainer Node-RED itu sendiri, bukan ke kontainer Mosquitto yang menjalankan broker MQTT. Karena kedua layanan berjalan pada kontainer yang berbeda, Node-RED tidak dapat menemukan broker melalui alamat localhost. Oleh karena itu, diperlukan alamat atau nama layanan yang sesuai agar komunikasi antar kontainer dapat berjalan.

  ii. Jelaskan bagaimana mekanisme *Docker Embedded DNS Server* dan *Bridge Network* bekerja sehingga kontainer node-red dapat menemukan kontainer mqtt-broker cukup dengan memanggil nama layanannya!

  Jawaban: Docker Embedded DNS Server berfungsi menerjemahkan nama layanan menjadi alamat IP internal kontainer. Ketika kontainer Node-RED dan Mosquitto berada dalam satu jaringan Bridge Network, keduanya dapat saling berkomunikasi menggunakan nama layanan. Dengan demikian, Node-RED cukup menggunakan nama mqtt untuk terhubung ke broker Mosquitto tanpa harus mengetahui alamat IP internal kontainer secara langsung.

- **Latihan 2 (Penanganan Masalah Konektivitas Jaringan & Subnet Firewall):**
  Seorang mahasiswa mencoba menghubungkan NodeMCU ESP8266 ke broker lokal di laptopnya. ESP8266 berhasil terhubung ke Wi-Fi dan mendapatkan alamat IP 192.168.43.102, sedangkan laptop memiliki alamat IP 192.168.1.15.

  i. Mengapa NodeMCU tidak akan pernah berhasil memublikasikan data ke broker Mosquitto pada kondisi tersebut?

Jawaban: NodeMCU tidak dapat terhubung dengan broker Mosquitto karena alamat IP NodeMCU 192.168.43.102 dan laptop 192.168.1.15 berada pada subnet yang berbeda dengan asumsi menggunakan subnet mask /24 atau 255.255.255.0. Kondisi tersebut menyebabkan kedua perangkat tidak berada dalam jaringan lokal yang sama. Akibatnya, NodeMCU tidak dapat langsung menjangkau broker MQTT pada laptop tanpa adanya konfigurasi routing yang sesuai.

ii. Langkah teknis apa yang harus dilakukan mahasiswa tersebut agar kedua perangkat berada dalam domain *broadcast / subnet* yang sama?

Jawaban: Langkah yang perlu dilakukan adalah menghubungkan laptop dan NodeMCU ke jaringan Wi-Fi atau hotspot yang sama. Setelah itu, pastikan keduanya memperoleh alamat IP dalam subnet yang sama. Contohnya, laptop menggunakan alamat IP 192.168.43.15, sedangkan NodeMCU menggunakan 192.168.43.102. Selanjutnya, periksa alamat IP laptop menggunakan perintah ipconfig, pastikan firewall mengizinkan koneksi melalui port MQTT 1883, kemudian gunakan alamat IP laptop sebagai alamat broker pada program NodeMCU.

- **Latihan 3 (Implementasi Filter Histeresis pada Rule Engine):**
  Pada sistem kendali dua posisi (*Bang-Bang*), jika suhu berfluktuasi cepat di sekitar ambang batas 31.0°C(misal: 31.05°C → 30.95°C → 31.02°C), lampu LED indikator akan berkedip-kedip cepat (*flickering / hunting*) yang mengganggu stabilitas visual sistem.
  Rancang modifikasi alur Node-RED menggunakan **function node** berbahasa JavaScript yang menerapkan logika **Histeresis**:
  - LED baru menyala jika suhu > 31.5°C(msg.payload = {"led": 1}).
  - LED baru padam jika suhu mendingin hingga < 30.0°C(msg.payload = {"led": 0}).
    Tuliskan kode JavaScript yang Anda masukkan ke dalam *Function Node* tersebut!

    Jawaban:

```
let led = context.get("led") || 0;

let suhu = Number(msg.payload);

if (suhu > 31.5) {
   led = 1;
} else if (suhu < 30.0) {
   led = 0;
}

context.set("led", led);

msg.payload = {
   led: led
};

return msg;
```

    Berdasarkan kode tersebut, LED akan menyala apabila suhu lebih dari 31,5°C dan akan padam apabila suhu kurang dari 30°C. Jika suhu berada di antara 30°C hingga 31,5°C, status LED akan tetap mengikuti kondisi sebelumnya. Dengan menerapkan

logika histeresis, perubahan suhu kecil tidak langsung mengubah kondisi LED sehingga sistem menjadi lebih stabil dan mengurangi terjadinya flickering.

- **Latihan 4 (Analisis Mekanisme Volume Mount & Persistensi Data):**
  Pada berkas docker-compose.yml, terdapat pemetaan direktori - ./nodered:/data dan - ./mosquitto/data:/mosquitto/data.

  i. Apa yang akan terjadi pada alur kanvas visual Node-RED yang telah Anda buat jika kontainer dimatikan dengan perintah docker compose down, kemudian dinyalakan kembali, seandainya baris volume tersebut dihapus dari docker-compose.yml?

  Jawaban: Jika konfigurasi volume ./nodered:/data dihapus, data Node-RED hanya tersimpan di dalam sistem penyimpanan kontainer. Ketika kontainer dihapus menggunakan perintah docker compose down, data konfigurasi yang tersimpan di dalam kontainer dapat ikut terhapus. Akibatnya, ketika kontainer dijalankan kembali, alur visual atau flow Node-RED yang telah dibuat berisiko hilang dan harus dikonfigurasi ulang.

  Hal yang sama berlaku pada pemetaan ./mosquitto/data:/mosquitto/data. Tanpa pemetaan tersebut, data Mosquitto tidak tersimpan pada direktori laptop sehingga berisiko hilang ketika kontainer dihapus.

  ii. Jelaskan perbedaan mendasar antara *Bind Mount* dan *Named Volume* pada Docker!

  Jawaban: Bind Mount merupakan metode penyimpanan yang menghubungkan direktori atau file tertentu pada komputer host dengan direktori di dalam kontainer. Lokasi penyimpanannya ditentukan langsung oleh pengguna. Contohnya adalah ./nodered:/data.

  Sementara itu, Named Volume merupakan metode penyimpanan yang dikelola oleh Docker. Pengguna cukup memberikan nama volume tanpa harus menentukan lokasi penyimpanan secara langsung. Contohnya adalah nodered_data:/data.

  Perbedaan utamanya terletak pada pengelolaan lokasi penyimpanan. Bind Mount menggunakan direktori yang ditentukan pengguna, sedangkan Named Volume menggunakan ruang penyimpanan yang dikelola oleh Docker. Keduanya dapat digunakan untuk menjaga persistensi data agar tetap tersedia meskipun kontainer dihentikan atau dihapus, selama media penyimpanannya tidak ikut dihapus.

7.7 Tugas: Pengamanan Server IoT Industri (MQTT Authentication & Access Control)

Pada tahap ini dilakukan pengamanan MQTT Broker lokal menggunakan sistem autentikasi username dan password. Sebelumnya, broker masih mengizinkan perangkat terhubung tanpa menggunakan kata sandi. Oleh karena itu, dilakukan perubahan konfigurasi agar hanya perangkat yang memiliki kredensial yang benar yang dapat mengakses broker. Pengamanan ini diterapkan pada NodeMCU ESP8266 dan Node-RED agar komunikasi telemetri serta kendali LED tetap berjalan setelah autentikasi diaktifkan.

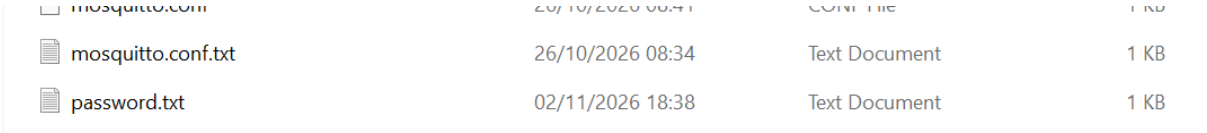
1. Membuat Berkas Kredensial Terenkripsi pada Kontainer Mosquitto
   Langkah pengerjaan:
   1. Pastikan Docker Desktop sudah berjalan.
   2. Pastikan container mqtt-broker dalam keadaan Up.
   3. Buka terminal atau PowerShell pada laptop.
   4. Jalankan perintah berikut:

```
D:\iot.server>docker exec -it mqtt-broker mosquitto_passwd -c /mosquitto/config/password.txt admin
Password:
Reenter password:
Adding password for user admin

What's next:
    Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug mqtt-broker
    Learn more at https://docs.docker.com/go/debug-cli/

D:\iot.server>
```

   5. Setelah berhasil, buka folder:



Pada tahap ini dibuat berkas kredensial untuk menyimpan username dan password yang akan digunakan dalam proses autentikasi MQTT Broker. Pembuatan berkas dilakukan menggunakan perintah mosquitto_passwd yang dijalankan langsung di dalam kontainer Mosquitto. Username yang digunakan adalah admin, sedangkan password yang ditetapkan adalah rahasia123. Password tersebut akan disimpan dalam bentuk hash sehingga tidak ditampilkan sebagai teks biasa. Berkas password.txt nantinya digunakan oleh Mosquitto sebagai sumber pemeriksaan kredensial setiap kali perangkat mencoba

2. Memperbarui Konfigurasi Kebijakan Keamanan Mosquitto Langkah pengerjaan:

1. Buka folder iot-server/mosquitto/config/.

2. Cari file mosquitto.conf.

3. Klik kanan file tersebut, lalu buka menggunakan Notepad atau Visual Studio Code.

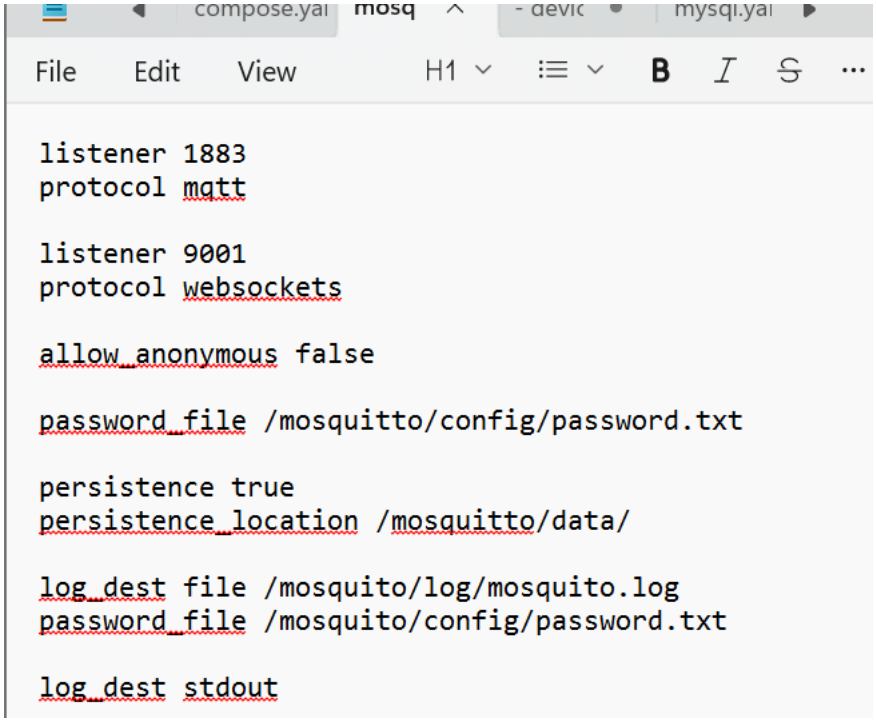
4. Cari konfigurasi: allow_anonymous true

5. Ubah menjadi: allow_anonymous false

6. Tambahkan baris berikut: password_file /mosquitto/config/password.txt

7. Pastikan konfigurasi akhirnya menjadi seperti berikut: listener 1883 allow_anonymous false password_file /mosquitto/config/password.txt log_dest stdout

8. Simpan file mosquitto.conf.



Pada tahap ini dilakukan perubahan kebijakan keamanan pada Mosquitto Broker dengan menonaktifkan akses anonim melalui pengaturan allow_anonymous false. Konfigurasi tersebut menyebabkan setiap perangkat yang ingin terhubung ke broker wajib menggunakan username dan password yang telah terdaftar. Selain itu, ditambahkan pengaturan password_file untuk menentukan lokasi berkas kredensial yang sebelumnya telah dibuat. Dengan konfigurasi ini, Mosquitto akan memeriksa username dan password sebelum mengizinkan perangkat melakukan komunikasi melalui MQTT.

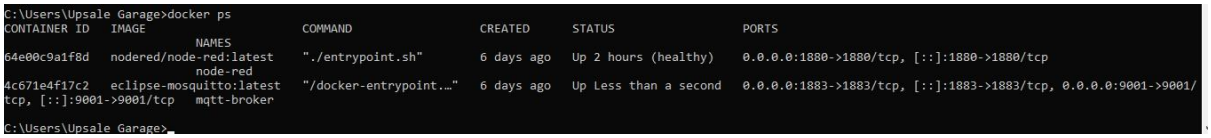
3. Memuat Ulang Layanan MQTT Broker

Langkah pengerjaan:

1. Buka PowerShell.
2. Jalankan perintah: docker restart mqtt-broker
3. Tekan Enter.
4. Tunggu hingga proses restart selesai.
5. Periksa kembali status kontainer dengan perintah:

docker ps

6. Pastikan kontainer mqtt-broker kembali dalam keadaan berjalan atau Up.



Setelah konfigurasi keamanan diperbarui, Mosquitto Broker perlu dimuat ulang agar perubahan kebijakan autentikasi dapat diterapkan. Proses restart dilakukan menggunakan perintah docker

restart mqtt-broker. Setelah kontainer kembali berjalan, broker akan menggunakan konfigurasi baru yang mewajibkan setiap klien memiliki kredensial yang valid sebelum dapat terhubung.

4. Mengamati Dampak Penolakan Koneksi Tanpa Autentikasi

Langkah pengerjaan:

1. Buka Node-RED melalui browser dengan alamat:

http://localhost:1880

2. Amati status node MQTT IN dan MQTT OUT pada kanvas.
3. Perhatikan apakah status koneksi berubah menjadi disconnected atau connection refused.
4. Selanjutnya, buka Arduino IDE.
5. Hubungkan NodeMCU ESP8266 ke laptop.
6. Buka Serial Monitor.
7. Atur baud rate sesuai program, yaitu 115200.
8. Amati pesan koneksi MQTT yang muncul.

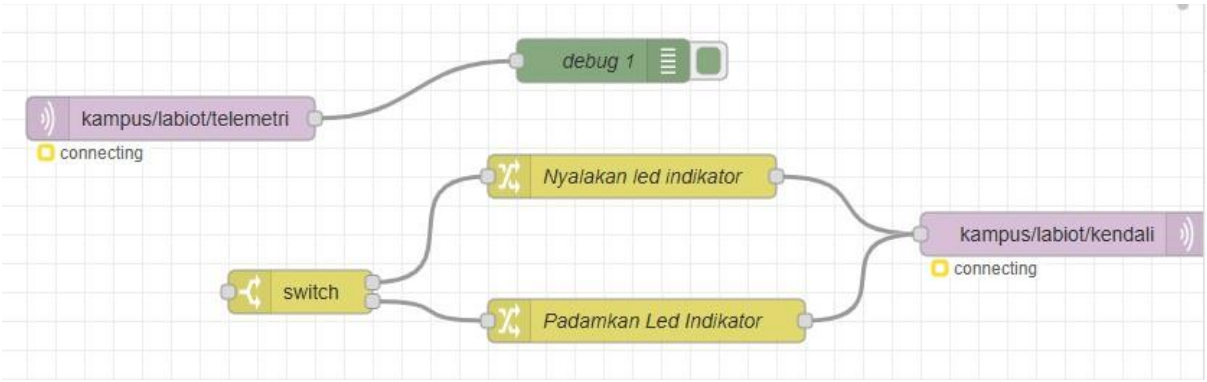
Contoh pesan yang diharapkan:

[MQTT] Menghubungkan ke Broker Lokal...

Gagal (rc=5). Coba lagi dalam 4 detik...

Mengapa terjadi penolakan koneksi?

Karena Mosquitto Broker sudah menerapkan autentikasi dan menolak klien yang mencoba terhubung tanpa username dan password yang benar. Node-RED dan ESP8266 sebelumnya masih menggunakan konfigurasi koneksi tanpa kredensial sehingga broker menolak permintaan koneksi tersebut.



Pada tahap ini dilakukan pengamatan terhadap dampak penerapan autentikasi pada MQTT Broker. Setelah akses anonim dinonaktifkan, Node-RED dan NodeMCU ESP8266 yang belum menggunakan username dan password tidak dapat terhubung kembali ke broker. Hal tersebut ditunjukkan dengan perubahan status koneksi MQTT pada Node-RED menjadi disconnected serta munculnya kode kesalahan rc=5 pada Serial Monitor ESP8266. Kode tersebut menunjukkan bahwa koneksi ditolak

karena klien tidak memiliki otorisasi yang sesuai. Kondisi ini membuktikan bahwa kebijakan autentikasi broker telah bekerja dalam membatasi akses perangkat yang tidak memiliki kredensial.

5. Memperbarui Autentikasi pada Program NodeMCU ESP8266

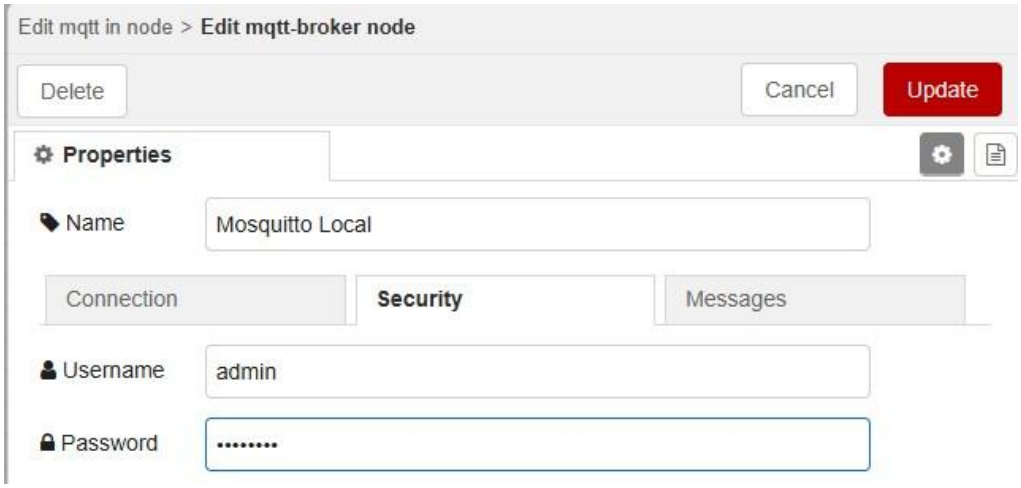
Langkah pengerjaan:

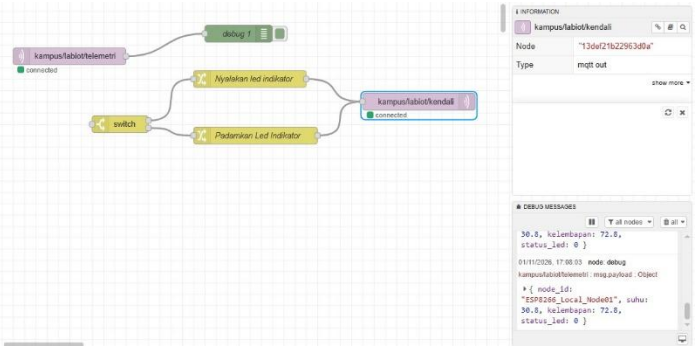
1. Buka program ESP8266 pada Arduino IDE.
2. Cari fungsi reconnectMQTT().
3. Pastikan username dan password sama dengan yang dibuat pada Mosquitto.
4. Klik Verify untuk memeriksa program.
5. Jika tidak ada kesalahan, hubungkan ESP8266 ke laptop.
6. Klik Upload untuk mengunggah program.
7. Buka Serial Monitor dengan baud rate 115200.
8. Amati hasil koneksi MQTT.

6. Memperbarui Kredensial Autentikasi pada Node-RED

Langkah pengerjaan:

1. Buka kembali Node-RED melalui browser: http://localhost:1880
2. Pada kanvas, klik dua kali node mqtt in.
3. Pada bagian Server, klik ikon pensil di samping Mosquitto Local.
4. Buka tab Security pada jendela konfigurasi broker.
5. Isi kredensial berikut: username dan password
6. Klik Update untuk menyimpan perubahan konfigurasi broker.
7. Klik Done pada pengaturan node MQTT IN.
8. Jika node MQTT OUT menggunakan konfigurasi broker Mosquitto Local yang sama, kredensialnya juga akan diperbarui.
9. Klik tombol merah Deploy.
10. Amati status MQTT IN dan MQTT OUT pada kanvas.





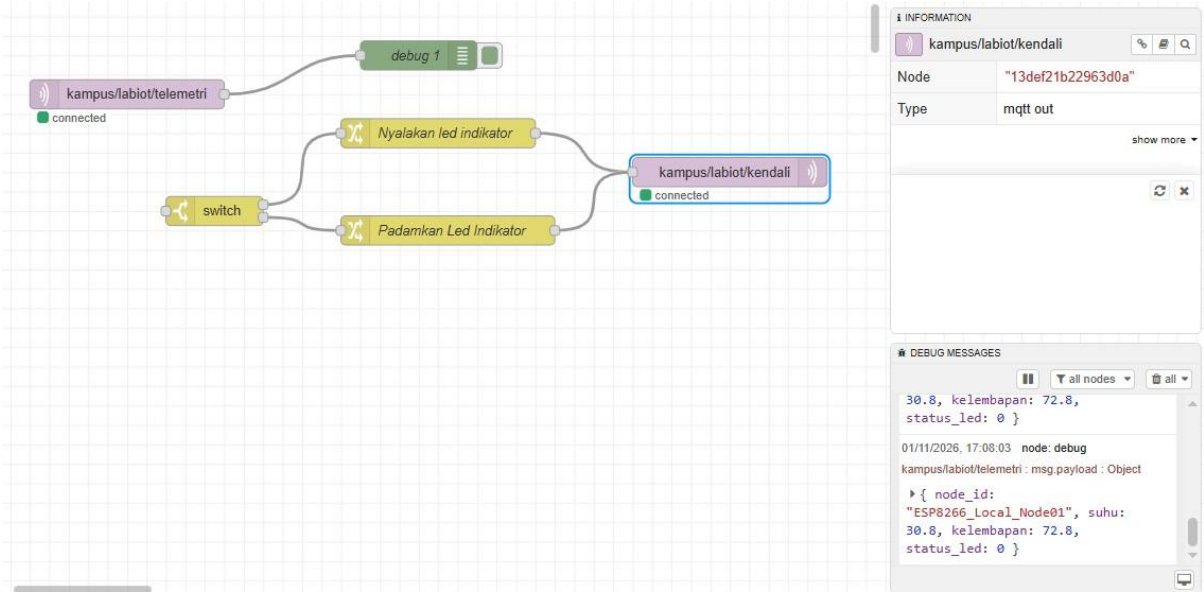
Pada tahap ini dilakukan pembaruan konfigurasi autentikasi pada Node-RED agar dapat kembali terhubung dengan Mosquitto Broker. Pengaturan dilakukan melalui konfigurasi server Mosquitto Local dengan memasukkan username dan password pada tab Security. Setelah perubahan disimpan dan Deploy dilakukan, Node-RED akan mencoba menghubungkan kembali node MQTT IN dan MQTT OUT menggunakan kredensial yang telah dimasukkan. Jika autentikasi berhasil, status koneksi akan kembali menjadi connected sehingga proses penerimaan data telemetri dan pengiriman perintah kendali dapat berjalan kembali.

7. Menguji Kembali Komunikasi Telemetri dan Kendali LED

Langkah pengerjaan:

1. Pastikan ESP8266 sudah terhubung menggunakan username dan password yang benar.
2. Pastikan Node-RED sudah menunjukkan status connected.
3. Buka panel Debug Messages pada Node-RED.
4. Amati apakah data telemetri kembali diterima.
5. Pastikan data node_id, suhu, kelembapan, dan status_led muncul.
6. Selanjutnya, periksa apakah flow Rule Engine dari praktikum 7.5.4 masih aktif.
7. Amati perubahan suhu sensor DHT22.
8. Jika suhu lebih dari 31°C, periksa apakah LED menyala.
9. Jika suhu kurang dari atau sama dengan 31°C, periksa apakah LED padam.

10. Pastikan perintah kendali tetap dikirim melalui topik kampus/labiot/kendali.



Setelah autentikasi diterapkan pada ESP8266 dan Node-RED, dilakukan pengujian ulang terhadap komunikasi telemetri dan sistem kendali otomatis. Hasil pengujian diamati melalui panel Debug Messages untuk memastikan data sensor kembali diterima oleh Node-RED. Selanjutnya, sistem Rule Engine diuji untuk memastikan perintah menyalakan dan mematikan LED tetap berjalan berdasarkan batas suhu yang telah ditentukan. Jika seluruh proses berjalan dengan baik, berarti sistem IoT lokal telah berhasil menggunakan autentikasi tanpa mengganggu fungsi monitoring dan kendali otomatis.